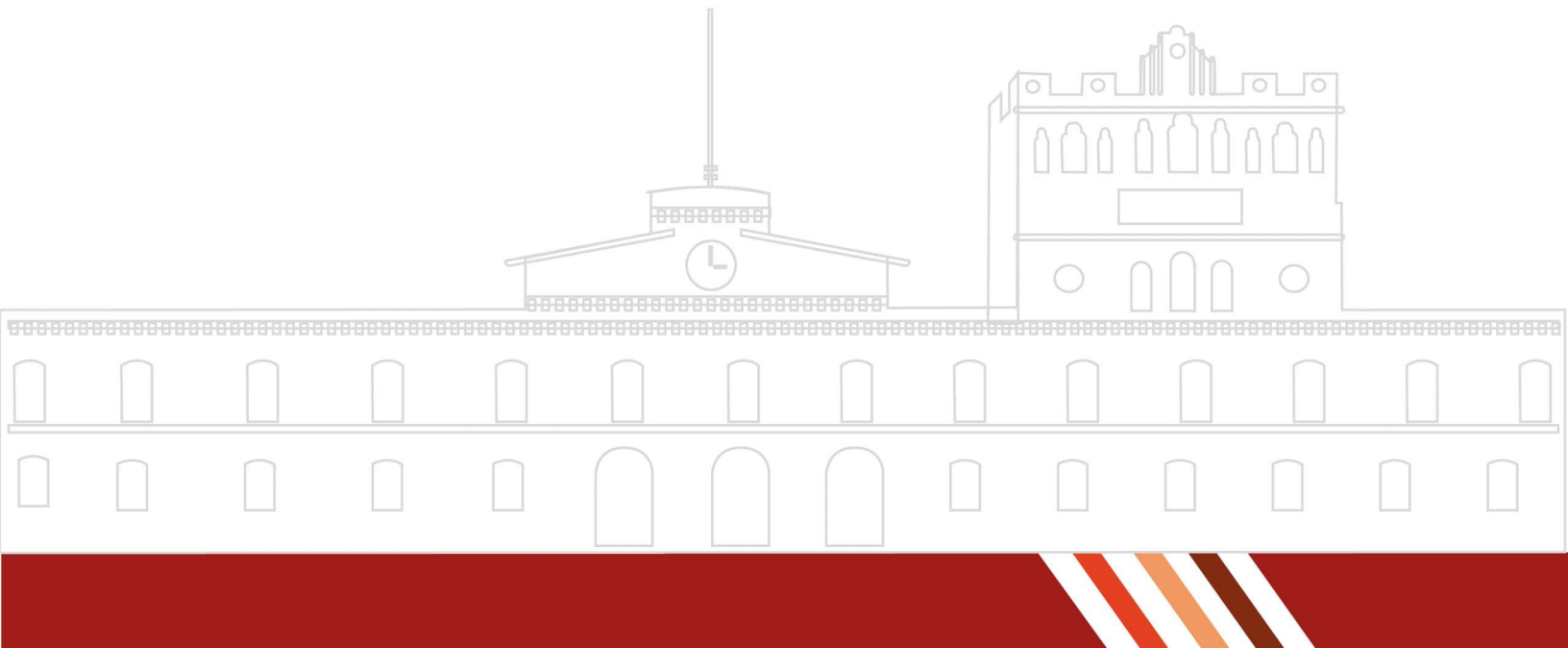


REPORTE DE PRÁCTICA NO. 3

CONSTRUCCIÓN DE UN AUTÓMATA FINITO DETERMINISTA

ALUMNO: Ortiz Salinas Samuel

Docente: Dr. Eduardo Cornejo-Velázquez



1. Identificación

- Nombre de la práctica: Construcción de un Autómata Finito Determinista
- No. de práctica: 3
- No. de sesiones: 2
- No. de integrantes máximo por equipo: 2

2. Introducción

Un Autómata Finito Determinista (AFD) es una máquina que va cambiando de estado según los símbolos que va leyendo, y al final dice si la cadena que recibió es válida o no para cierto lenguaje [1]. En la práctica pasada armamos varios AFD a mano, con su diagrama y su tabla de transiciones. En esta práctica ya no se diseña el autómata, sino que se recibe uno ya definido (sus estados, su alfabeto, sus transiciones y sus estados finales) como datos de entrada, y lo que se pide es programar algo que simule ese autómata para cualquier cadena que se le dé. Esto es parecido a lo que hace un compilador cuando reconoce las palabras de un lenguaje de programación [2].

3. Marco teórico

Un **Autómata Finito Determinista (AFD)** se define como una tupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- Σ es el alfabeto de entrada
- Q es el conjunto finito y no vacío de los estados
- δ es la función de transición, $\delta : Q \times \Sigma \rightarrow Q$
- $q_0 \in Q$ es el estado inicial
- $F \subset Q$ es un conjunto de estados finales de aceptación ($F \neq \emptyset$)

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea $w = w_1 w_2 \dots w_n$ una cadena de símbolos donde $w_i \in \Sigma$. Entonces, A **acepta** w si existe una secuencia de estados $r_0, r_1, \dots, r_n \in Q$ con tres condiciones:

1. $r_0 = q_0$: se empieza en el estado inicial.
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ para $i = 0, 1, \dots, n-1$: se va cambiando de estado según lo que diga δ .
3. $r_n \in F$: al final, se acepta la cadena solo si el estado donde se quedó es de aceptación.

Entonces, A **reconoce el lenguaje** L si $L = \{w \mid A \text{ acepta } w\}$.

Para programar δ no hace falta usar una matriz enorme: basta con guardar cada transición $\delta(q_i, x) = q_j$ en algo como un diccionario, donde la llave es el par (estado, símbolo) y el valor es a dónde se va. Así, revisar una cadena completa es nada más ir buscando en ese diccionario símbolo por símbolo [3].

4. Objetivos

Objetivo general

Construir un programa en un lenguaje de programación de alto nivel para implementar un Autómata Finito Determinista (AFD) que permita analizar un conjunto de cadenas de símbolos (W) para determinar si pertenecen o no al lenguaje (L) reconocido por el AFD.

Objetivos específicos

- Identificar los componentes de un Autómata Finito Determinista (AFD).
- Implementar un AFD en un lenguaje de programación de alto nivel.
- Determinar si diferentes conjuntos de símbolos (W) pertenecen al lenguaje (L) reconocido por el AFD.

5. Equipo y software a utilizar

Equipo de cómputo con software para programación a elección de los alumnos. Se usó C++ (con el estándar C++17), el compilador GCC, y el framework Qt 6 (Widgets) para hacerle una interfaz gráfica al programa, editable con Qt Designer. El proyecto se construye con CMake.

6. Desarrollo

Diseño de la solución

Para representar el AFD se usaron dos estructuras nada más: los estados finales F se guardan en un `std::set<int>` y las transiciones δ en un `std::map<std::pair<int, char>, int>`, donde la llave es (estado origen, símbolo) y el valor es el estado destino. El alfabeto Σ no se guarda aparte, porque igual ya está implícito en los símbolos que aparecen en las transiciones. Para saber si una cadena se acepta, se recorre símbolo por símbolo desde q_0 y se va consultando el mapa; si en algún momento no hay transición para ese símbolo, la cadena se rechaza ahí mismo, y si se llega al final, se acepta solo si el estado en el que se quedó está en F .

Listing 1: Funcion que revisa si el AFD acepta una cadena (afd.cpp).

```
1 bool aceptaAFD(const std::string& cadena, int q0,
2               const std::map<std::pair<int, char>, int>& delta,
3               const std::set<int>& finales) {
4     int estado = q0;
5
6     for (char simbolo : cadena) {
7         auto transicion = delta.find({estado, simbolo});
8
9         if (transicion == delta.end())
10            return false;
11
12        estado = transicion->second;
13    }
14    return finales.count(estado) > 0;
15 }
```

Como la interfaz solo deja evaluar hasta 5 cadenas, no hizo falta usar un `std::vector` ni ninguna estructura extra para guardar los resultados: basta un arreglo de tamaño fijo `std::string[5]`, igual de simple que las 5 casillas de la interfaz. La función `ejecutarAFD` arma las transiciones y los finales, y en el mismo ciclo en el que revisa cada cadena va agregando el resultado a un solo texto que después se muestra en pantalla.

Listing 2: Evaluacion de las hasta 5 cadenas, sin vector (afd.cpp).

```
1 std::string salida;
2 for (int i = 0; i < cantidadCadenas && i < 5; i++) {
3     const std::string& cadena = cadenas[i];
4
5     salida += cadena;
6     salida += aceptaAFD(cadena, q0, delta, finales) ? " -> ACEPTADA\n" : " -> RECHAZADA\n";
7 }
```

Interfaz gráfica con Qt Widgets

Se hizo una ventana en **Qt Widgets** (editable desde **Qt Designer**, archivo `mainwindow.ui`) para no tener que escribir a mano todo el bloque de texto de entrada que pide la práctica. La ventana tiene:

- Seis `QSpinBox` para N , S , D , q_0 , T y C (número de cadenas, tope de 5).
- Dos campos de texto para el alfabeto Σ y los estados finales F .
- Un área de texto para las transiciones, una por línea (**origen simbolo destino**).
- Cinco campos de texto fijos para las cadenas (`cadena1` a `cadena5`); solo se usan las primeras C .

El código quedó separado en dos archivos: `afd.h/afd.cpp` con la lógica del AFD (sin nada de Qt) y `mainwindow.h/mainwindow.cpp` con la ventana. Hay dos botones: **Ejecutar**, que junta los datos de todos los campos y muestra el resultado, y **Limpiar**, que solo borra todo sin ejecutar nada.

Como la práctica no pide validar cada dato por separado, todo lo que hace el botón Ejecutar está metido en un solo `try/catch`. Si algo sale mal (por ejemplo una transición mal escrita), se lanza una excepción y se muestra un solo mensaje de error al final, en vez de estar revisando cada campo antes de usarlo.

Listing 3: Boton Ejecutar con manejo de error unico al final (mainwindow.cpp).

```
1 void MainWindow::onEjecutar() {
2     ui->salidaEdit->clear();
3
4     try {
5         int q0 = ui->spinQ0->value();
6         int cantidadCadenas = ui->spinC->value();
7
8         QLineEdit *camposCadena[5] = {
9             ui->cadena1, ui->cadena2, ui->cadena3, ui->cadena4, ui->cadena5};
10
11         std::string cadenas[5];
12         for (int i = 0; i < 5; i++)
13             cadenas[i] = camposCadena[i]->text().toStdString();
14
15         std::string salida = ejecutarAFD(q0,
16                                         ui->lineAlfabeto->text().toStdString(),
17                                         ui->lineFinales->text().toStdString(),
18                                         ui->transicionesEdit->toPlainText().toStdString(),
19                                         cadenas,
20                                         cantidadCadenas);
21
22         ui->salidaEdit->setPlainText(QString::fromStdString(salida));
23     } catch (const std::exception &e) {
24         QMessageBox::critical(this, "Error",
25                               QString("Ocurrio un error al procesar la entrada:\n%1").arg(e.what()));
26     }
27 }
```

Ejecución y validación

Para probar el programa se usó el ejemplo del manual: un AFD con $N = 3$ estados, alfabeto $\Sigma = \{0, 1\}$, $q_0 = 1$, $F = \{2\}$, que acepta las cadenas con una cantidad par de 0's después del último 1. En la Figura 1 se ve la ventana con esos datos ya cargados, y en la Figura 2 el resultado después de darle clic a **Ejecutar**, con las cadenas 101, 0, 11 y 1000100001.

Identificación del AFD

N (numero de estados): 3

S (tamano del alfabeto): 2

D (numero de transiciones): 6

q0 (estado inicial): 1

T (numero de estados finales): 1

C (cantidad de cadenas, maximo 5): 4

Alfabeto y estados finales

Simbolos de Sigma (separados por espacio): 0 1

Estados finales F (separados por espacio): 2

Transiciones (una por linea: origen simbolo destino)

1	1	2
1	0	3
2	0	3
2	1	2

Cadenas a evaluar (maximo 5, se usan las primeras C)

Cadena 1: 101

Cadena 2: 0

Cadena 3: 11

Cadena 4: 1000100001

Cadena 5:

Ejecutar Limpiar

Resultado:

Figure 1: Ventana principal con la definición del AFD de ejemplo.

Identificación del AFD

N (numero de estados):

S (tamano del alfabeto):

D (numero de transiciones):

q0 (estado inicial):

T (numero de estados finales):

C (cantidad de cadenas, maximo 5):

Alfabeto y estados finales

Simbolos de Sigma (separados por espacio):

Estados finales F (separados por espacio):

Transiciones (una por linea: origen simbolo destino)

1	1	2
1	0	3
2	0	3
2	1	2

Cadenas a evaluar (maximo 5, se usan las primeras C)

Cadena 1:

Cadena 2:

Cadena 3:

Cadena 4:

Cadena 5:

Resultado:

```

101 -> ACEPTADA
0 -> RECHAZADA
11 -> ACEPTADA
1000100001 -> ACEPTADA

```

Figure 2: Resultado de evaluar las cuatro cadenas de ejemplo.

Los cuatro resultados salen igual que en el manual: 101 y 11 se aceptan, 0 se rechaza y 1000100001 también se acepta.

También se probó qué pasa si se escribe mal una transición (por ejemplo, poner una línea de texto que no sigue el formato **origen simbolo destino**). En la Figura 3 se ve que el programa no intenta arreglarlo ni ignorarlo: deja que salte el error y muestra un solo mensaje al final, sin haber evaluado nada.

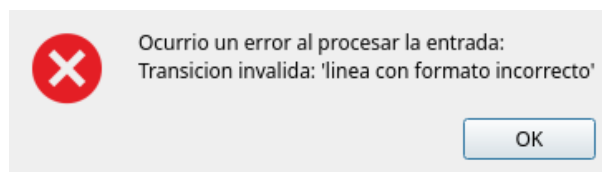


Figure 3: Mensaje de error mostrado ante una transición con formato inválido.

7. Resultados

Se logró hacer un programa que simula cualquier AFD que cumpla con los límites de la práctica (hasta 100 estados y 100 símbolos, y hasta 5 cadenas por evaluación desde la interfaz). La parte que revisa el AFD quedó separada de la parte gráfica, así que la misma función de la práctica se pudo usar igual de consola que en la interfaz de Qt, sin cambiarle nada. Con el ejemplo del manual el programa dio los mismos cuatro resultados esperados, y cuando se le dio una entrada mal escrita mostró un solo mensaje de error al final, sin necesidad de andar revisando cada dato antes de usarlo.

8. Conclusiones

Con esta práctica quedó más claro que un AFD no es solo un dibujo con círculos y flechas, sino que se puede representar con datos simples (un conjunto para F y un diccionario para δ) y con eso ya se puede programar un simulador que sirva para cualquier autómata, no solo para uno en específico. También ayudó separar la lógica del AFD de la parte de la ventana: la función que revisa si una cadena se acepta no tuvo que cambiar nada al pasar de leer datos de la consola a leerlos de los campos de la interfaz.

9. Cuestionario

1. **¿Qué paradigma de programación utilizaste para la implementación del AFD?, ¿Cuáles elementos del lenguaje de programación utilizaste?**

Para la parte del AFD se usó más que nada un estilo estructurado, con ciclos y condicionales. Para la ventana sí se usó orientado a objetos, porque Qt está hecho con clases y con señales y slots para conectar los botones. Del lenguaje se usaron `std::map`, `std::set` y `std::string`, y `try/catch` para el manejo de errores.

2. **¿Qué estructuras de datos utilizaste para implementar los componentes del AFD?**

Un `std::set<int>` para los estados finales y un `std::map<std::pair<int, char>, int>` para las transiciones. Para las cadenas a evaluar se usó un arreglo normal de tamaño 5, ya que la interfaz no deja meter más de esa cantidad.

3. **¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición?, ¿cuáles son las limitaciones del programa que construiste?**

El método `find` del `std::map` sirvió para buscar si existe la transición para un estado y un símbolo; si no la encuentra, la cadena se rechaza. Como limitaciones, el programa solo admite símbolos de un carácter, no revisa que el AFD esté completo (que existan todas las transiciones posibles) ni que los números que se pusieron en N , S , D y T sean congruentes con lo que realmente se escribió, y solo deja evaluar 5 cadenas a la vez.

4. **¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD?**

Que los estados sean números, que cada símbolo del alfabeto sea un solo carácter, que q_0 sea uno de esos estados, que F no esté vacío, y que las transiciones se escriban en el formato **origen simbolo destino**. No hace falta que el AFD esté completo: si falta una transición, el programa simplemente rechaza la cadena en ese punto en vez de marcar error.

Referencias Bibliográficas

References

- [1] Giró, J., Vázquez, J., Meloni, B., Constable, L. (**2015**). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
- [2] Ruiz Catalán, J. (**2010**). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
- [3] Brookshear, J. G. (**1995**). *Teoría de la Computación. Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.